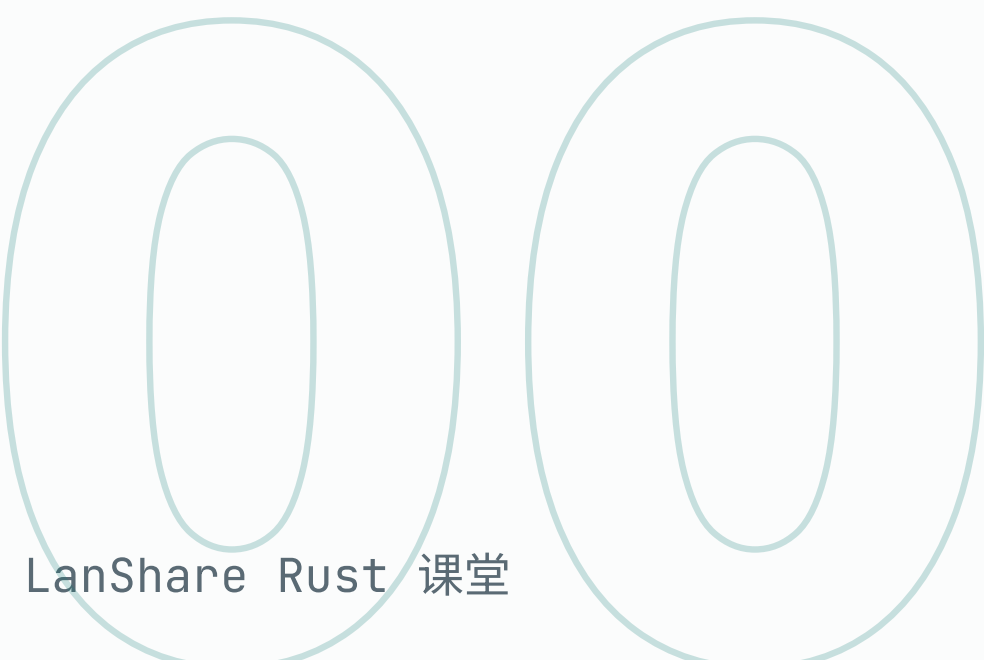


第 0 集 · 约 16 分钟

写给会 Python 的人

同一个程序写了两遍，拿它来学 Rust

配套阅读：书的第 0 章



这一集你会学到

1. 为什么说 Python 在运行时检查，Rust 在编译时检查
2. 异常 vs Result、None vs Option，一一对上号
3. 一个让 Python 程序员必踩的坑：len() 数的不是同一个东西
4. 所有权的第一印象：值会“搬家”

先看个结果

同一个软件，Python 版 9.3 MB、启动 0.64 秒；Rust 版 2.3 MB、启动 0.044 秒。

这还不是重点。重点是：Rust 版在编译时就拦下了好几类 Python 要到运行时才暴露的 bug。

体积和启动速度（实测）

v1 Python + PyInstaller

每次启动先解压到临时目录



640 毫秒（启动到可以访问）

v2 Rust

直接运行机器码



44 毫秒（启动到可以访问）

开发机 i9-12900H 实测，5 次取中位数（数据见 docs/bench.md）

一句话记住最大的区别

Python 让你先跑起来，错了再报；Rust 先逼你把各种情况想清楚。

Python 在运行时检查
类型错了、忘了处理 None、多线程同时改数据……跑到那一行才炸

Rust 在编译时检查
这些问题大多编译都过不了

所以写 Rust 时“编译器老报错”是正常的
它把运行时的问题提前到了你面前

概念对照表

Python	Rust	本项目里的例子
<code>pip install + requirements.txt</code>	<code>cargo add + Cargo.toml</code>	依赖管理
<code>list</code> / <code>dict</code> / <code>set</code>	<code>Vec</code> / <code>HashMap</code> / <code>HashSet</code>	会话表、文件 id 表
<code>None</code>	<code>Option<T> : None / Some(x)</code>	按 id 找文件名
抛异常 <code>raise</code>	返回 <code>Result<T, E></code>	解密失败
<code>with open(...)</code>	变量离开作用域自动释放	文件、锁
类 + 方法	<code>struct</code> + <code>impl</code>	Storage、AppState
<code>asyncio</code>	<code>tokio</code>	服务端
<code>unittest</code>	<code>#[test] + cargo test</code>	120 个测试

异常 vs Result

Python v1 · textboard.py

```
1 def add(self, text):
2     if not isinstance(text, str) or not text.strip():
3         raise ValueError("文字不能为空")
4     if len(text) > MAX_TEXT_LENGTH:
5         raise ValueError("文字超过 %d 字" % MAX_TEXT_LENGTH)
6     with self._lock:
7         item = {"id": self._next_id, "text": text, "time":
```

Rust v2 · textboard.rs

```
1 pub fn add(&self, text: &str) -> Result<TextItem, &'static Error> {
2     if text.trim().is_empty() {
3         return Err("empty_text");
4     }
5     if text.chars().count() > MAX_TEXT_CHARS {
6         return Err("text_too_long");
7     }
8     let mut guard = self.inner.lock().expect("文字板的锁不会为其他线程持有");
```

动手：len() 到底数的是什么

输入一段文字

局域网快传 LanShare

Python: len(s)

14

数“字符” (Unicode 码点)

Rust: s.len()

24

数 UTF-8 字节

Rust: s.chars().count()

14

数字符，和 Python 的 len 一样

每个字符在 UTF-8 里占几个字节

局	域	网	快	传		L	a	n	S	h	a	r	e
e5 b1 80	e5 9f 9f	e7 bd 91	e5 bf ab	e4 bc a0	20	4c	61	6e	53	68	61	72	65

None vs Option

Python

```
1 name = self._by_id.get(file_id)  # 可能是 None
2 if name is None:
3     return error("not_found")
4 use(name)  # 忘了上面的检查, 这里就炸
```

Rust v2 · transfer.rs

```
1 let Some(name) = st.files.lock().expect("锁不会中毒").name_
2     return down_error("not_found");
3 };
4 // 到这里 name 一定是 String, 不可能是“空”
```

预告：值会“搬家”

```
1 let s1 = String::from("hi");
2 let s2 = s1;           // 搬家
3 println!("{}", s1);    // 还想用 s1?
4 let s3 = &s2;           // 借用：只看不拿
5 }                       // 作用域结束
```

变量（栈上）
(空)

数据（堆上）
已释放

离开作用域：主人 s2 被清理，它拥有的“hi”跟着释放。不需要垃圾回收。

对比 Python: `b = a` 只是给同一个对象多起一个名字，`a` 照样能用。

异步：谁是“惰性”的

第 5 集我们会因为这个踩一个坑

| **Python** `coro = fetch(u)`：只是创建了协程，还没执行

| **Rust** `let fut = client.get(u).send();`：只是创建了 Future，还没执行

| **JavaScript** `const p = fetch(u);`：Promise 一创建就已经开始发请求

选择题

Python 的 `len("局域网")` 是 3，Rust 的 `"局域网".len()` 是多少？

- A 3
- C 6

- B 9
- D 编译不过

选择题

Python 的 `len("局域网")` 是 3，Rust 的 `"局域网".len()` 是多少？

A 3

3 是字符数。Rust 的 `len()` 数的是 UTF-8 字节。

B 9

一个汉字在 UTF-8 里占 3 个字节，3 个字就是 9 个字节。要数字符用 `chars().count()`。

C 6

那是 UTF-16 的算法（每个汉字 2 字节），Rust 的字符串是 UTF-8。

D 编译不过

能编译。`len()` 是合法的，只是数的东西和 Python 不一样。

选择题

下面这段 Python 搬到 Rust 时，`isinstance` 检查还需要吗？

```
1 def add(self, text):  
2     if not isinstance(text, str):  
3         raise ValueError()
```

A 需要，Rust 也可能传进来别的类型

B 不需要，类型在签名里写死了

C 需要，但要改成 match

选择题

下面这段 Python 搬到 Rust 时，`isinstance` 检查还需要吗？

```
1 def add(self, text):  
2     if not isinstance(text, str):  
3         raise ValueError()
```

A 需要，Rust 也可能传进来别的类型

不会。函数签名写着 `text: &str`，传别的类型编译不过。

C 需要，但要改成 `match`

`match` 用来处理枚举，不是用来检查参数类型的。

B 不需要，类型在签名里写死了

对。Rust 的类型检查发生在编译时，调用方只能传字符串进来。

选择题

Python 的 `asyncio` 协程和 Rust 的 Future，下面哪个说法对？

A 两者都是惰性的，要 `await` 或交给运行时才执行

B 两者都是一创建就开始执行

C Python 惰性，Rust 立即执行

选择题

Python 的 `asyncio` 协程和 Rust 的 Future，下面哪个说法对？

A 两者都是惰性的，要 `await` 或交给运行时才执行

对。调用 `async` 函数只是创建了一个“还没开始的计算”。

B 两者都是一创建就开始执行

那是 JavaScript 的 `Promise`。

C Python 惰性，Rust 立即执行

Rust 的 `Future` 也是惰性的，不 `poll` 就什么都不做。

想一想

Python 的 `with open(path) as f:` 在 Rust 里对应什么？为什么 Rust 不需要 with 语法？

想一想

Python 的 `with open(path) as f:` 在 Rust 里对应什么？为什么 Rust 不需要 `with` 语法？

对应 `let f = File::open(path)?;`。f 离开所在的大括号时自动关闭文件。Rust 的所有资源（文件、锁、网络连接）都遵循“离开作用域就释放”，叫 RAII，所以不需要专门的 `with`。

这一集我们学了

- 🚫 Python 运行时检查，Rust 编译时检查：编译器报错是在帮你
- 🚫 异常 → Result, None → Option，都写在类型里，不处理编译不过
- 🚫 Rust 的 len() 数 UTF-8 字节，数字符要 chars().count()
- 🚫 值会“搬家”：所有权是 Rust 独有的概念

下一集

下一集：cargo 和 workspace，把项目骨架搭起来，顺便学会读编译器的报错